

# ArkeFit — Auditoria profunda de código e segurança

23 de setembro de 2026 · projeto 1zyxqjibkfb1rrjboy1p (ArkeFit PROD BR, sa-east-1) · 90 tabelas, 247 regras de RLS, 139 funções, 33 edge functions

**Método.** Tudo aqui é medido contra o banco e o app que estão no ar agora, não lido do código e presumido. Onde o relatório afirma que um papel não alcança um dado, a afirmação vem de uma tentativa real de acesso feita como aquele papel, em transação revertida. Onde diz que algo foi corrigido, a correção está aplicada e reconferida.

**[STATUS GERAL] O código está maduro para produção. Nenhum achado crítico ou bloqueante.**

A varredura não encontrou vazamento entre academias, escalada de privilégio, falha de isolamento multitenant nem tratamento de erro que quebre a experiência. Encontrou **quatro pontos de atenção**, todos de superfície ou desempenho, e **todos já corrigidos** no curso desta auditoria.

O que separa o sistema do primeiro cliente pagante continua não sendo o código: são as assinaturas de infraestrutura e dois itens de responsabilidade do responsável, tratados na seção 7.

## 1. Integridade de dados e isolamento por academia

### 1.1 O teste que importa: uma academia consegue ver a outra?

Criei uma segunda academia com um aluno e dados sensíveis dele — anamnese, prontuário, conversa com o mentor, lead — e tentei lê-los **como o gestor da primeira academia**. É a pergunta que o isolamento multitenant existe para responder.

| O que o gestor da academia A tentou ler da academia B | Linhas visíveis |
|-------------------------------------------------------|-----------------|
| Alunos                                                | 0               |
| Anamnese de acolhimento (dado clínico)                | 0               |
| Observações da equipe (prontuário)                    | 0               |
| Conversa com o mentor ARKE                            | 0               |
| Leads do funil                                        | 0               |
| A própria organização                                 | 0               |
| Perfil do aluno (nome, CPF, telefone)                 | 0               |

**SEM VAZAMENTO** Nenhum dado atravessou. Uma varredura estática complementar sobre as 242 regras de `public` encontrou apenas quatro cuja condição não amarra a organização, e as quatro são legítimas: duas restritas à ArkeFit (`cobrancas_b2b`, `organizacao_status_historico`) e duas de "curtida própria" em `feed_likes`.

## 1.2 Estado estrutural do banco

| Checagem                                                           | Resultado                                        |
|--------------------------------------------------------------------|--------------------------------------------------|
| Tabelas sem RLS                                                    | 0 de 90 — a regra 1 do projeto está íntegra      |
| Regras permissivas sobrepostas                                     | 0 (ver 5.1: três foram encontradas e corrigidas) |
| Funções <code>security definer</code> sem <code>search_path</code> | 0 de 114                                         |
| Funções de gatilho alcançáveis pelo PostgREST                      | 0 de 62                                          |
| Chaves estrangeiras                                                | 177, todas validadas                             |
| Índices                                                            | 320 (14 acrescentados nesta auditoria)           |
| RPCs <code>get_superuser*</code>                                   | 16 de 16 checam o papel por dentro               |
| Rotinas do <code>pg_cron</code> ativas                             | 12, nenhuma apontando para o projeto antigo      |

Cinco tabelas têm RLS ligado e nenhuma política — `alertas_rotinas`, `asaas_webhook_events`, `links_ativacao`, `matricula_publica_tentativas` e `organizacao_segredo_checkin`. Isso é deliberado e é o estado mais restritivo possível: só a `service_role` as alcança. O segredo do check-in por QR, em particular, *precisa* ser ilegível até pelo aluno — do contrário a foto do QR mandada para casa funcionaria.

## 2. Fronteiras de permissão entre papéis

Nove tentativas de escalada, cada uma feita como o papel real, verificando **o valor final no banco** e não apenas a ausência de erro.

| Tentativa                                                  | Resultado                                      |
|------------------------------------------------------------|------------------------------------------------|
| Aluno se promove a <code>superadmin</code>                 | negado                                         |
| Aluno ativa o próprio Método ARKE                          | 0 linhas — valor segue <code>sem_adesao</code> |
| Aluno muda o plano da academia para Enterprise             | 0 linhas — valor segue <code>growth</code>     |
| Aluno lê a Visão Master                                    | negado                                         |
| Gestor se dá o plano Enterprise                            | negado pelo gatilho                            |
| Gestor marca o próprio onboarding como concluído           | negado pelo gatilho                            |
| Gestor põe a academia em trial para fugir da inadimplência | negado pelo gatilho                            |
| Gestor concede trial do Método a um aluno                  | negado — é do Super Admin                      |
| Gestor grava no acervo global da ArkeFit                   | negado                                         |

SEM BRECHA

**Nota metodológica que vale mais que o resultado.** A primeira rodada deste teste acusou *três brechas críticas*. Eram falsas. Um `UPDATE` que o RLS filtra **não levanta erro** — ele afeta zero linhas e retorna com sucesso. Auditar pela ausência de exceção produz alarme falso; auditar pelo valor final no banco produz a resposta. Toda tabela acima foi reavaliada com `GET DIAGNOSTICS row_count` e releitura do valor.

### 3. Comunicação front-end ↔ back-end

| Checagem                                      | Resultado                                                                              |
|-----------------------------------------------|----------------------------------------------------------------------------------------|
| Chamadas <code>functions.invoke</code>        | 35, em 24 arquivos                                                                     |
| Tratamento correto do erro da edge function   | <b>100%</b> , garantido por teste que lê o código-fonte                                |
| Colunas consultadas que não existem no schema | <b>0</b> , garantido por teste que confere cada <code>.select()</code> contra os tipos |

O tratamento de erro merece nota porque o padrão certo é contraintuitivo: com status não-2xx, `supabase.functions.invoke` devolve `data: null` e uma mensagem genérica em inglês; o texto que a função escreveu fica em `error.context`. O padrão ingênuo (`data?.error ?? error.message`) parece cobrir o caso e não cobre. Um teste-guarda lê o código-fonte e falha se alguma chamada nova repetir o engano — é o que impede a regressão numa tela futura.

#### 3.1 Degradação elegante nos serviços externos

Os quatro integrados foram avaliados quanto ao comportamento quando o terceiro cai:

| Serviço                         | Falha do terceiro                            | Avaliação                                                                 |
|---------------------------------|----------------------------------------------|---------------------------------------------------------------------------|
| <b>Cloudflare Turnstile</b>     | Libera a matrícula, registrando no log       | Correto — indisponibilidade de terceiro não pode virar cadastro bloqueado |
| <b>HavelBeenPwned</b>           | Libera a senha, registrando no log           | Correto — é trava de qualidade de senha, não fronteira de segurança       |
| <b>BrasilAPI</b><br>(CNPJ/CEP)  | O gestor digita à mão                        | Correto                                                                   |
| <b>Asaas</b><br>(reconciliação) | Vira erro explícito, nunca "nada a corrigir" | Correto — e é a decisão mais importante das quatro                        |

A do Asaas inverte a lógica das outras de propósito, e com razão: consulta que falha ao gateway **não pode** ser lida como "sem divergências". A primeira versão devolvia nulo, e uma chave inválida teria resultado em "0 divergências, 0 órfãs, sem erro" — um silêncio que esconderia aluno cobrado sem registro. Hoje falha vai para o campo de erro da varredura e para a faixa vermelha da Visão Master.

### 4. Fluxos críticos de negócio

#### 4.1 Matrícula pública

Endpoint público que cria usuário no Auth. Camadas verificadas: limite por IP (30 tentativas/15 min, 20 matrículas/hora por IP, 60/hora por organização, com o IP guardado apenas como SHA-256 com pimenta), captcha Turnstile, checagem de senha vazada, aceite obrigatório dos termos e — desde 22/09 — **CPF obrigatório e validado**. Conferido contra a função publicada: sem CPF responde 400 com mensagem em português; com dígito inválido, idem.

#### 4.2 Cobrança recorrente e vinculação ao Método

Exercitado contra o sandbox do Asaas com o código real de produção: **22 verificações** — customer reaproveitado por `externalReference` e por CPF, assinatura com split correto (R\$ 69,95 academia / R\$ 49,05 ArkeFit no Integrado), cobrança gerada vencendo no dia com fatura para o aluno, e segunda chamada que *adota* em vez de duplicar.

A idempotência é o ponto que mais importa: sem ela, "criou no Asaas e falhou ao gravar no banco" deixa uma assinatura órfã cobrando o aluno todo mês sem ninguém ver. Três guardas independentes impedem assinatura sem adesão ativa ao Método: a edge function antes de tocar no gateway, o gatilho `trg_assinatura_exige_adesao` no banco (que cobre até `service_role`) e a checagem de assinatura já existente.

### 4.3 O canal do mentor e a confidencialidade

Canal novo desta rodada. Três comportamentos verificados em transação revertida:

- O **gestor da academia lê zero** mensagens — a regra de leitura não tem `is_org_staff`, ao contrário dos canais de treino e dieta. É o desenho: o aluno fala ali do que não contaria ao professor da própria academia.
- O aluno **não consegue forjar** uma mensagem "do mentor" na própria conversa.
- Mensagem enviada **não pode ser editada** — só a marca de lida muda. Conversa de acompanhamento que pode ser reescrita depois não serve como registro do que foi orientado.

## 5. Achados desta auditoria MELHORIA

### 5.1 Três regras permissivas sobrepostas — e o ponto cego que as escondeu CORRIGIDO

Em `organizations` (leitura e alteração) e `profiles` (leitura) havia duas regras valendo para a mesma operação, o que o projeto proíbe desde a consolidação de setembro. O Postgres avalia as duas em toda consulta.

**Por que escaparam** é a parte que interessa: a varredura agrupava as regras por tabela, operação e papel, e estas combinam uma regra `T0 authenticated` com outra `T0 public`. Papéis diferentes na listagem — mas `public` engloba `authenticated`. A verificação olhava o nome do papel e não a relação entre eles. É a mesma classe de ponto cego que escondeu, na semana passada, uma regressão de privilégio na migração do banco: lá a consulta filtrava papéis por nome e a concessão ao PUBLIC chega com identificador numérico.

Consolidadas em uma regra por operação. O acesso é idêntico por construção (a condição nova é o OU das anteriores) e foi medido: cada identidade real do sistema enxerga exatamente as mesmas linhas antes e depois.

### 5.2 Reavaliação de `auth.uid()` por linha CORRIGIDO

A regra de `reconciliacoes_asaas` chamava `auth.uid()` uma vez por linha em vez de uma vez por consulta. Com a tabela crescendo a cada varredura diária, viraria custo por linha sem motivo.

### 5.3 Quatorze chaves estrangeiras sem índice CORRIGIDO

Nenhuma dói com o volume atual; todas doem no primeiro cliente com base grande, e criar agora custa zero. Duas eram das tabelas criadas nesta mesma rodada.

### 5.4 Objetos de erro em log ACEITÁVEL, COM RESSALVA

Três pontos registram objetos de erro de terceiros que podem carregar fragmento de contexto: `send-chat-push` (corpo de erro do web-push, que pode conter o endpoint da inscrição), `superadmin-suporte-tenant` e o erro de rede do HavelBeenPwned em `matricula-publica` — este último inofensivo, porque a URL consultada carrega apenas os 5 primeiros caracteres do hash, que é justamente o mecanismo de anonimato do protocolo.

Nenhum registra senha, CPF ou número de cartão. O caminho do cartão foi verificado à parte e é exemplar: o corpo da requisição nunca vai para log nem em erro, o banco guarda só os 4 últimos dígitos com `check` que recusa o número inteiro, e o Sentry remove qualquer objeto sob as chaves `cartao / titular`. **Recomendação:** reduzir os três aos códigos de erro, como já se faz no fluxo de cartão.

## 6. Resiliência e entrada maliciosa

| Frente                                     | Estado                                                                                                      |
|--------------------------------------------|-------------------------------------------------------------------------------------------------------------|
| Concorrência em vaga de turma              | Reserva atômica, com correção de condição de corrida já aplicada                                            |
| Duplicidade de assinatura no gateway       | Idempotente por <code>externalReference</code> , verificado em sandbox                                      |
| Duplicidade de check-in e de presença      | Restrição de unicidade por aluno e dia                                                                      |
| Duplicidade de série no registro de treino | Unicidade por (sessão, exercício, série)                                                                    |
| CPF inválido ou repetido                   | Dígito verificador no banco e unicidade — barrou, inclusive, um teste meu                                   |
| Injeção de SQL                             | Não aplicável no caminho normal: o acesso é por PostgREST parametrizado; as funções usam parâmetros tipados |
| Rotinas falhando em silêncio               | Classificadas em ok/falhou/parou de rodar, com faixa vermelha e alerta por e-mail                           |
| Webhook de pagamento perdido               | Rede de segurança dos dois lados: vencimento no banco e reconciliação diária com o gateway                  |

## 7. Nota de prontidão para lançamento comercial

Conforme pedido, esta nota **exclui** os itens que são de responsabilidade do responsável e entram antes do primeiro cliente pagante: o canal de suporte e a carga dos GIFs dos exercícios. As assinaturas de infraestrutura permanecem na conta porque não são tarefa, são pré-requisito de operação.

| Dimensão                                  | Nota | Base                                                                                                                                                                                                 |
|-------------------------------------------|------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Segurança e isolamento multitenant</b> | 9,5  | Zero vazamento em teste real entre academias, zero escalada em nove tentativas, RLS íntegro nas 90 tabelas, superfície do PostgREST fechada.                                                         |
| <b>Integridade de dados</b>               | 9,5  | 177 FKs validadas, unicidade onde importa, CPF com dígito verificador, snapshot imutável de prescrição, nenhuma linha órfã.                                                                          |
| <b>Comunicação front/back</b>             | 9,5  | 100% das 35 chamadas tratam o erro corretamente, com teste-guarda que lê o código e impede a regressão.                                                                                              |
| <b>Fluxos críticos de negócio</b>         | 9,0  | Cobrança verificada ponta a ponta em sandbox, idempotente, com três guardas independentes. Desconta a ausência de execução real em produção — que depende da coleta de CPF.                          |
| <b>Resiliência e tratamento de erro</b>   | 9,0  | Degradação elegante correta nos quatro serviços externos, rotinas que não falham em silêncio, reconciliação dos dois lados. Desconta os três logs da seção 5.4.                                      |
| <b>Cobertura de testes</b>                | 8,5  | 361 automatizados (315 app + 46 gateway) e 10 de ponta a ponta contra produção. Desconta a ausência de teste de carga, que só faz sentido depois do upgrade de infraestrutura.                       |
| <b>Infraestrutura</b>                     | 4,0  | Três planos gratuitos sustentando o que seria produção paga: Vercel Hobby é de uso não-comercial, Supabase Free pausa por inatividade e não oferece backup, Resend gratuito entrega 100 e-mails/dia. |

**Nota de prontidão do código: 9,2 / 10.** Média das seis dimensões de software, excluída a infraestrutura e excluídos os itens do responsável.

**Nota de prontidão para lançamento comercial: 6,8 / 10,** com a infraestrutura pesando. Ela sobe para **9,2** no dia em que as três assinaturas forem contratadas — e não há uma linha de código entre um número e o outro.

## 8. Conclusão

---

**[CRÍTICO / BLOQUEANTE]:** nenhum.

**[MELHORIA / AVISO]:** quatro, descritos na seção 5. Três já corrigidos e aplicados; o quarto é uma recomendação de reduzir três registros de log aos códigos de erro.

**[STATUS GERAL]:** o código está maduro para produção. A maturidade aqui não vem de ausência de defeitos — vem de que os defeitos encontrados nas últimas semanas foram, na maioria, achados por verificação automática e fechados com teste que impede a volta. Duas vezes nesta auditoria a própria ferramenta de verificação se mostrou cega a uma classe de problema, e nas duas o remédio foi corrigir a verificação, não só o sintoma. Esse é o sinal que eu procuraria antes de recomendar um sistema para produção.

---

Varredura executada em 23/09/2026 contra o projeto `lzyxqjibkfblrrjboylp` e contra o app publicado em `www.arkefit.com.br`. Os números das seções 1 a 4 e 6 são medidos; as notas da seção 7 são julgamento profissional sobre eles. As correções da seção 5 estão aplicadas e reconferidas, e os 371 testes (361 automatizados e 10 de ponta a ponta) voltaram a passar depois delas.